

PC-Based Smart Home Automation System

Engineering Design, Embedded Control, Serial Communication and Desktop HMI Documentation

Author: Hailemariam Muche

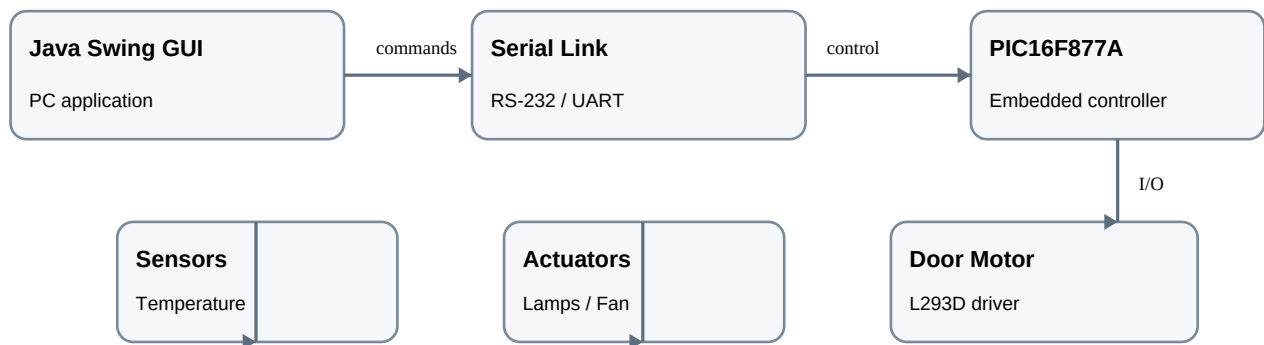
Discipline: Electrical and Computer Engineering

Repository: <https://github.com/Hailemariam-Muche/PC-Based-Smart-Home-Automation-System>

Document purpose. This document records the engineering concept, system architecture, hardware/firmware organization, Java desktop application, serial protocol, control logic, simulation approach, testing considerations and repository structure of the project.

1. Executive Overview

The PC-Based Smart Home Automation System is a practical embedded-and-software integration project in which a Java desktop application acts as the human-machine interface for a PIC16F877A-based controller. Commands are transmitted through a serial interface, interpreted by the microcontroller, and converted into appliance or actuator actions. The embedded controller also returns temperature and device-status information to the PC, allowing the application to present live feedback.



The project demonstrates an important engineering principle: a useful automation system is not only a microcontroller circuit. It is a coordinated system in which the user interface, communication protocol, embedded firmware, sensors, actuators and simulation environment operate as one engineered solution.

2. Project Objectives

- Develop a centralized PC interface for controlling household loads.
- Establish reliable serial communication between a Java application and a PIC16F877A.
- Control three lamps independently and operate a motorized door.
- Measure and display temperature at the PC.
- Implement automatic fan control according to a defined temperature threshold.
- Return meaningful status messages from the embedded controller to the desktop application.
- Use Proteus simulation to support circuit development and verification.
- Organize the complete project into reproducible hardware, firmware, software, simulation and documentation resources.

3. System Architecture

The architecture is divided into two major layers. The PC layer provides the user interface and communication management. The embedded layer provides real-time I/O control, temperature processing and actuator control. The serial link forms the boundary between these layers.

Layer	Main elements	Responsibility
PC / HMI	Java, Swing, NetBeans, jSerialComm	User commands, connection management, GUI status and logging
Communication	RS-232 / UART	Bidirectional command and status exchange
Embedded controller	PIC16F877A, MikroC firmware	Command decoding, I/O control, temperature processing
Actuation	3 lamps, fan, door motor, L293D	Physical response to controller decisions
Simulation	Proteus Design Suite	Circuit verification and development support

4. Hardware Design

Component	Function
PIC16F877A	Central embedded controller
Three lamp outputs	Independent appliance/load control
Fan motor	Temperature-based automatic cooling output
Door motor	Open/close mechanical actuation
L293D	Bidirectional motor-driver interface for door motor
Temperature sensor	Environmental temperature measurement
RS-232 serial interface	PC-to-microcontroller communication
PC	Supervisory control and graphical user interface

The design separates logic-level control from motor actuation. The microcontroller provides the control signals, while the motor-driver stage handles the door motor interface. This is a fundamental embedded-systems design practice because inductive loads should not be driven directly from microcontroller I/O pins.

5. Embedded Firmware

The PIC16F877A firmware is developed in MikroC PRO for PIC. Its responsibilities include initializing the controller, interpreting serial commands, switching appliance outputs, controlling the door motor, acquiring temperature information, applying the fan-control rule and transmitting status information back to the PC.

The project therefore uses a simple command-and-feedback architecture: the PC sends a concise command, the embedded controller performs the requested action, and the controller returns a status message that the Java application can use to update its display.

6. Serial Communication Protocol

The documented communication parameters are 9600 baud, 8 data bits, no parity and 1 stop bit (8N1). The serial channel is bidirectional.

Parameter	Value
Baud rate	9600 bps
Data bits	8
Parity	None
Stop bits	1
Direction	PC → PIC commands; PIC → PC feedback

6.1 Command Set

Command	Action
A	Lamp 1 ON
a	Lamp 1 OFF
B	Lamp 2 ON
b	Lamp 2 OFF
C	Lamp 3 ON
c	Lamp 3 OFF
O	Door OPEN

Command	Action
X	Door CLOSE

6.2 Feedback Messages

Message family	Meaning
TEMP:value	Temperature measurement returned to the PC
L1_ON / L1_OFF	Lamp 1 state
L2_ON / L2_OFF	Lamp 2 state
L3_ON / L3_OFF	Lamp 3 state
FAN_ON / FAN_OFF	Fan state
DOOR_OPENING / DOOR_OPENED	Door opening sequence/status
DOOR_CLOSING / DOOR_CLOSED	Door closing sequence/status

Using explicit status tokens rather than relying only on button state makes the system more robust conceptually: the GUI can represent the state reported by the controller rather than assuming that every transmitted command was executed successfully.

7. Temperature Monitoring and Automatic Fan Control

Temperature is measured by the embedded controller and transmitted to the Java application for display. The documented automatic-control rule is:

Temperature $\geq 30\text{ }^{\circ}\text{C}$ $\rightarrow$ Fan ON
Temperature $< 30\text{ }^{\circ}\text{C}$ $\rightarrow$ Fan OFF

The threshold is a design parameter. In a real installation, hysteresis could be introduced—for example, turning the fan on at a higher threshold and off at a slightly lower threshold—to prevent rapid switching around the boundary.

8. Java Desktop Application

The PC application is implemented in Java using Swing and is intended to provide a straightforward supervisory dashboard. The repository identifies NetBeans as the development environment and jSerialComm as the serial communication library.

GUI function	Engineering role
COM-port selection	Select the serial interface connected to the controller
Connect / disconnect	Manage communication state
Lamp controls	Issue independent appliance commands
Door controls	Issue open/close commands
Temperature display	Present controller measurement
Fan status	Present automatic/manual state
Communication log	Provide diagnostic visibility
Device feedback	Reflect embedded-system responses

9. Development Workflow

The project follows a sensible layered development sequence. Separating the circuit, firmware, communication protocol and desktop application reduces integration complexity and makes faults easier to isolate.

Phase	Engineering activity
1. System concept	Define controllable loads, feedback and user interface.
2. Circuit design	Develop and simulate the PIC-based hardware in Proteus.
3. Firmware	Implement I/O control, UART and temperature logic in MikroC.
4. Communication	Define and test command/feedback messages.
5. Desktop HMI	Develop Java Swing controls and status displays.
6. Integration	Connect Java application to the serial interface.
7. Functional test	Verify appliances, door, temperature and fan logic.
8. Debugging	Use logs, simulation and hardware observations to isolate faults.

10. Proteus Simulation

Proteus provides a controlled environment for validating the embedded circuit before physical deployment. It is particularly useful for checking microcontroller pin assignments, motor-driver logic, serial communication behavior and firmware-driven output changes.

A good engineering workflow is to validate one subsystem at a time—first controller and outputs, then the motor-driver stage, then temperature acquisition, then serial communication, and finally the complete system.

11. Testing and Verification Strategy

Test	Expected result
Power-up initialization	Controller starts in a known and safe output state.
Lamp 1 command	Lamp 1 changes state and matching feedback is returned.
Lamp 2 command	Lamp 2 changes state and matching feedback is returned.
Lamp 3 command	Lamp 3 changes state and matching feedback is returned.
Door open/close	Motor operates in the intended direction and status transitions are reported.
Temperature acquisition	Valid temperature value reaches the PC display.
Fan threshold	Fan turns on at/above 30 °C and off below 30 °C.
Serial disconnect	GUI handles loss of connection without unsafe actuator assumptions.
Integrated operation	GUI state and controller-reported state remain consistent.

12. Engineering Strengths

- Clear separation between supervisory software and real-time embedded control.
- Bidirectional serial communication rather than one-way appliance control.
- Explicit device-status feedback for improved observability.
- Combination of simulation, firmware and desktop application development.
- Simple command protocol that is easy to inspect, test and extend.
- Practical demonstration of integrating legacy embedded hardware with modern desktop software.

13. Engineering Improvements and Future Work

- Introduce framed messages or checksums for stronger serial communication integrity.
- Add acknowledgement/error messages for commands that cannot be executed.

- Introduce command timeouts and safe-state handling if the PC connection is lost.
- Add hysteresis to temperature-based fan control.
- Separate protocol, GUI and serial-driver responsibilities more formally in the Java application.
- Add configurable temperature thresholds through the GUI.
- Extend the system with additional sensors and loads while preserving a documented command namespace.
- For a modern deployment, consider galvanic isolation and appropriate protection when interfacing with mains-powered appliances.

14. Repository Structure

The current repository is organized into dedicated directories, which is appropriate for a portfolio-quality engineering project.

Directory	Purpose
Demonstration/	Demonstration material and evidence
Documentation/	Technical project documentation
Hardware/	Hardware design, connections and related evidence
Java_Application/	Java desktop application source
MikroC_Firmware/	PIC16F877A embedded firmware
Proteus_Simulation/	Proteus circuit/simulation files
Serial_Communication/	Communication-related material and implementation

Keeping these layers separate makes the repository useful not only as a project archive but also as an engineering reference for future maintenance or extension.

15. Reproduction Guide

- Clone or download the repository and inspect the README and directory structure.
- Open the Proteus project and verify the documented hardware connections.
- Compile the PIC firmware using the appropriate MikroC environment.
- Program the microcontroller or load the compiled firmware into the simulation.
- Build/open the Java application in the intended Java development environment and ensure jSerialComm is available.
- Connect the PC to the controller using the appropriate serial interface.
- Select the correct COM port and verify the 9600-8N1 configuration.
- Test individual loads before performing integrated operation.
- Use the communication log and returned status tokens to diagnose integration problems.

16. Safety and Practical Deployment

The repository represents a prototype/engineering demonstration rather than a certified residential electrical installation. If lamps or other loads represent mains-powered appliances, appropriate isolation, fusing, enclosures, earthing, creepage/clearance and locally applicable electrical-safety requirements must be addressed before practical deployment. The low-voltage controller should never be treated as a substitute for a properly rated mains switching stage.

17. Project Significance

This project is a strong example of systems engineering because it crosses the traditional boundary between embedded hardware and application software. The PIC16F877A performs deterministic low-level control, while Java provides a richer human interface and diagnostic layer. The serial protocol connects these domains and creates a complete, understandable control system.

18. Repository Reference

Primary GitHub repository: <https://github.com/Hailemariam-Muche/PC-Based-Smart-Home-Automation-System>

The repository should be regarded as the authoritative source for the current implementation, source files, simulation assets and project evidence. This document explains the engineering intent and architecture surrounding those files.

Prepared as an engineering project record

Hailemariam Muche — Electrical and Computer Engineering